



Laboratorio de Datos
UNAL - Sede de La Paz

CV4E

GUIA #1

INSTALACIÓN Y CONFIGURACIÓN DE ENTORNO DE DESARROLLO

IDE - ENV - VENV - DEVC



Índice

1. Entorno de desarrollo y herramientas	1
1.1. Instalación de Visual Studio Code	1
1.2. Instalación de Python	2
1.3. Instalación de Git	2
1.4. Configurar la terminal por defecto en VSCode	3
2. Tipos de entornos	4
2.1. Entornos virtuales	4
2.2. Contenedores	4
2.3. Máquinas virtuales	5
2.4. ¿Cuál usar?	5
3. Primeros pasos en la Terminal	6
3.1. Abrir la terminal y entender el prompt	6
3.2. Navegación por el sistema de archivos	6
3.3. Preparativos: instalar uv	7
3.4. Manos a la obra: tu primer proyecto	7

1. Entorno de desarrollo y herramientas

Si nunca has programado, es normal que palabras como *IDE*, *terminal* o *Git* suenen intimidantes. La buena noticia es que solo son herramientas de trabajo: igual que el microscopio, la pipeta o la bitácora de campo, cada una cumple una función concreta y, después de usarlas un par de veces, dejan de dar miedo.

En esta sección montaremos tu “Espacio de trabajo” paso a paso. Estas son las cuatro piezas que instalaremos y para qué sirve cada una:

- **Editor de código:** el programa donde escribiremos nuestros análisis. Es como un procesador de texto, pero diseñado para código: colorea las palabras según su función (**resaltado de sintaxis**), autocompleta lo que escribes y te avisa de errores antes de ejecutar.
- **Python:** el lenguaje de programación que usaremos durante el curso. Es uno de los más utilizados en ciencia por su sintaxis sencilla y su enorme colección de bibliotecas para análisis de datos.
- **Git:** un **sistema de control de versiones**: una bitácora automática que registra cada cambio en tus archivos, quién lo hizo y cuándo. Olvídate de carpetas tipo `analisis_final_v3_AHORA_SI.py`.
- **Terminal:** una ventana donde damos instrucciones al computador escribiendo comandos de texto. La exploraremos a fondo en la sección 3.

Cuando un editor de código integra todas estas herramientas en un solo lugar (editor + depurador + control de versiones + terminal), se le llama **entorno de desarrollo integrado** o *IDE*, por sus siglas en inglés. Existen varios (PyCharm, Eclipse, Atom), pero en este curso usaremos **Visual Studio Code** (VSCode): es gratuito, de código abierto, rápido y muy popular en la comunidad científica, lo que significa que casi cualquier duda que tengas ya fue respondida en internet.

1.1. Instalación de Visual Studio Code

Sigue estos pasos:

1. Visita el sitio web oficial: <https://code.visualstudio.com/>.
2. Haz clic en el botón de descarga que corresponda a tu sistema operativo (Windows, macOS o Linux).
3. Ejecuta el instalador descargado y acepta las opciones que vienen marcadas por defecto.

💡 **Consejo (Windows):** durante la instalación, marca la casilla **Agregar la acción Abrir con Code** cuando aparezca. Te permitirá abrir cualquier carpeta en VSCode con el clic derecho del ratón.

Un recorrido por la interfaz

Al abrir VSCode por primera vez verás varios paneles. No intentes memorizarlos todos ahora: la Imagen 1 te servirá de mapa para volver cada vez que lo necesites.

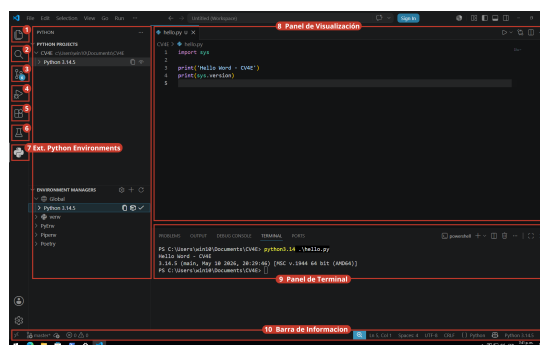


Imagen 1: Interfaz de Visual Studio Code con sus paneles numerados.

- **1. Explorador de archivos:** muestra las carpetas y archivos de tu proyecto y permite navegar entre ellos.
- **2. Buscador (lupa):** busca texto o archivos dentro de todo el proyecto.
- **3. Control de versiones:** muestra los cambios en tu código y permite gestionarlos con Git (lo instalaremos en la sección 1.3).
- **4. Depuración:** ejecuta el código paso a paso para encontrar y corregir errores.
- **5. Extensiones:** pequeños complementos que añaden funciones a VSCode, como instalar una app en el teléfono.
- **6. Tests:** ejecuta y muestra los resultados de las pruebas automáticas del código.
- **7. Extensión de Python:** aparecerá cuando la instalemos en la sección 1.2.
- **8. Editor de código:** el área principal donde se escribe el código.
- **9. Terminal integrada:** permite ejecutar comandos sin salir de VSCode.
- **10. Barra de estado:** información útil de un vistazo: errores, rama de Git activa, versión de Python, etc.

1.2. Instalación de Python

Para trabajar con Python en VSCode instalaremos la **extensión oficial de Python**, que añade resaltado de sintaxis, autocompletado, depuración y soporte para entornos virtuales (un concepto que veremos en la sección 2). Sigue estos pasos:

1. Haz clic en el icono de **Extensiones** (panel 5 de la Imagen 1) en la barra lateral izquierda.
2. En el campo de búsqueda escribe `ms-python.python` e instala la extensión oficial de Microsoft.
3. Junto con ella se instalarán automáticamente otras extensiones que mejoran la experiencia:
 - **Pylance** (`ms-python.vscode-pylance`): detección de errores y autocompletado inteligente.
 - **Python Debugger** (`ms-python.debugpy`): depuración de código.
 - **Python Environments** (`ms-python.python-environments`): gestión de entornos virtuales.
4. Reinicia VSCode para que los cambios surtan efecto.
5. Al volver a abrirlo, deberías ver el icono de Python en la barra lateral, como en el panel 7 de la Imagen 1. ✓ ¡Listo!

⚠ Atención: la extensión es el “traductor” entre VSCode y Python, pero también necesitas el lenguaje en sí. Si Python no está instalado en tu computador, la propia extensión te ofrecerá instalarlo automáticamente: acepta y deja que termine antes de continuar.

1.3. Instalación de Git

Git será nuestra bitácora de laboratorio digital: registra cada versión de tu trabajo y es la herramienta estándar para colaborar en proyectos de software y de ciencia reproducible. Para instalarlo:

1. Visita el sitio web oficial: <https://git-scm.com/downloads>.
2. Descarga el instalador correspondiente a tu sistema operativo.
3. Ejecuta el instalador y acepta las opciones por defecto.

Alternativamente, si usas Windows, puedes instalarlo escribiendo un solo comando en la terminal PowerShell:

PowerShell

```
1 | winget install --id Git.Git -e --source winget
```

Una vez instalado, preséntate ante Git: debes indicarle tu nombre y tu correo electrónico para que cada cambio que registres quede firmado por ti. Escribe estos dos comandos (con tus datos reales):

Terminal

```
1 | git config --global user.name "Tu Nombre"
2 | git config --global user.email "tu.email@ejemplo.com"
```

💡 **Consejo:** usa el mismo correo con el que piensas crear tu cuenta de GitHub, la plataforma donde se comparten proyectos gestionados con Git. Así tus contribuciones quedarán vinculadas a tu perfil.

1.4. Configurar la terminal por defecto en VSCode

Como último paso, configuraremos qué terminal abre VSCode por defecto. Junto a Git se instaló **Git Bash**, una terminal que en Windows ofrece una experiencia muy similar a la de Linux, el estándar en entornos de desarrollo y en servidores de cómputo científico. Para seleccionarla:

1. Abre la paleta de comandos de VSCode con `Ctrl` + `Shift` + `P`.
2. Escribe `Terminal: Select Default Profile` y pulsa `Enter` sobre la opción con ese nombre.
3. De la lista que aparece, elige **Git Bash**.

Con esto tu laboratorio digital está completo: editor, lenguaje, control de versiones y terminal. Antes de empezar a usarlos, en la siguiente sección veremos un concepto clave para mantener tus proyectos ordenados y reproducibles: los **entornos**.

2. Tipos de entornos

Imagina que en tu laboratorio cultivas dos organismos distintos en la misma placa de Petri: tarde o temprano uno contaminará al otro. En programación ocurre algo parecido. Los programas dependen de **bibliotecas** (paquetes de código ya escrito que reutilizamos), y cada proyecto puede necesitar versiones distintas de la misma biblioteca. Si instalamos todo “en la misma placa” —es decir, directamente en el sistema—, los proyectos terminan interfiriendo entre sí.

La solución es la misma que en el laboratorio: **aislar**. Para ello existen distintos tipos de entornos aislados, cada uno con un nivel de aislamiento diferente. Conocerlos te ayudará a elegir el adecuado según el proyecto. Veremos los tres más comunes, del más ligero al más completo.

2.1. Entornos virtuales

Un **entorno virtual** es una carpeta aislada dentro de tu proyecto donde se instalan las bibliotecas que ese proyecto necesita, con sus versiones exactas, sin afectar al resto del sistema ni a otros proyectos. Es como asignar a cada experimento su propia placa de Petri: cada proyecto tiene su “ecosistema” de bibliotecas limpio y controlado.

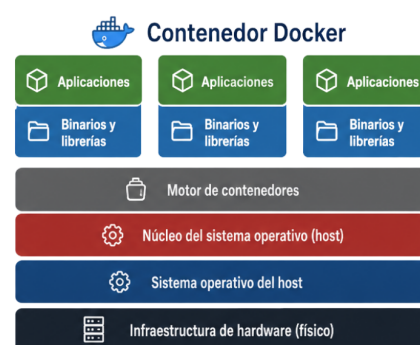


Imagen 2: Cada proyecto usa su propio entorno virtual, con sus propias versiones de las bibliotecas, sin interferir con los demás.

Este tipo de entorno es el más usado en el día a día con Python, donde las dependencias varían mucho entre proyectos. Además, facilita la **reproducibilidad**: si anotas qué bibliotecas (y versiones) usa tu entorno, cualquier colega podrá recrearlo y obtener tus mismos resultados.

2.2. Contenedores

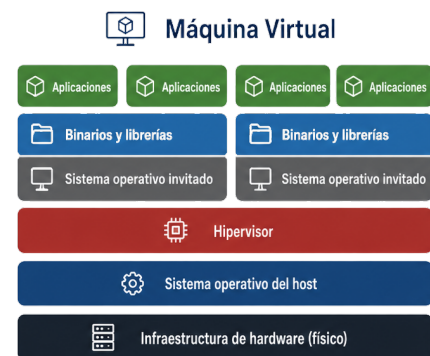
Un **contenedor** va un paso más allá: empaqueta tu aplicación junto con sus bibliotecas y una versión reducida del sistema operativo que necesita. Piensa en un kit de campo: una caja portátil con todo lo necesario para trabajar igual en cualquier sitio, sin depender de lo que haya en el laboratorio de destino. Su gran ventaja es la **portabilidad**: el contenedor se ejecuta exactamente igual en cualquier computador que tenga un motor de contenedores compatible, como **Docker**. Esto permite compartir aplicaciones con configuraciones específicas sin preocuparse por las diferencias entre sistemas operativos, y con un consumo de recursos mucho menor que el de una máquina virtual.



2.3. Máquinas virtuales

Una **máquina virtual** emula un computador completo, con su propio sistema operativo, dentro del tuyo. Es el máximo nivel de aislamiento: como construir una réplica entera de otro laboratorio dentro del tuyo.

Resulta útil para probar software en un sistema operativo distinto al que tienes (por ejemplo, ejecutar Linux dentro de Windows) o para trabajar en un entorno totalmente separado. A cambio, es la opción más pesada: ocupa varios gigabytes y consume bastante memoria y procesador.



2.4. ¿Cuál usar?

La Tabla 1 resume las diferencias entre los tres tipos:

	Entorno virtual	Contenedor	Máquina virtual
¿Qué aísla?	Las bibliotecas de un proyecto	La aplicación y un mini sistema operativo	Un computador completo
Peso	Ligero (megabytes)	Intermedio	Pesado (gigabytes)
Analogía	Una placa de Petri por experimento	Un kit de campo portátil	Una réplica completa del laboratorio
Uso típico	Trabajo diario con Python	Compartir aplicaciones entre sistemas	Probar otros sistemas operativos

Tabla 1: Comparación de los tres tipos de entornos aislados.

Para este curso usaremos **contenedores** y principalmente los **entornos virtuales**: son ligeros, fáciles de crear y cubren la necesidad principal, que es mantener las bibliotecas de cada proyecto ordenadas y reproducibles. En la siguiente sección aprenderás a usar la terminal y, como ejercicio final, crearás tu primer proyecto con su entorno virtual.

3. Primeros pasos en la Terminal

La **terminal** (también llamada línea de comandos o consola) es una ventana donde damos instrucciones al sistema operativo escribiendo comandos de texto, en lugar de hacer clic en botones. Al principio puede parecer austera —una pantalla con texto y un cursor parpadeando—, pero es una de las herramientas más poderosas que aprenderás: permite automatizar tareas repetitivas, gestionar proyectos y es la vía de acceso a los servidores de cómputo donde se ejecutan los análisis grandes, que no tienen interfaz gráfica.

Todos los sistemas operativos incluyen una terminal, aunque sus comandos varían un poco entre ellos. En Windows las predeterminadas son PowerShell y el Símbolo del sistema; en macOS y Linux se usa la terminal de Unix. Para que todos hablemos el mismo idioma, en el curso usaremos **Git Bash**, que ya instalamos junto a Git en la sección 1.3 y que ofrece en Windows los mismos comandos que Linux y macOS.

3.1. Abrir la terminal y entender el prompt

La forma más cómoda de abrir la terminal es desde VSCode:

- Usa el atajo `Ctrl+J` o el menú `Terminal > New Terminal`.

Como en la sección 1.4 configuramos Git Bash como perfil por defecto, verás algo parecido a esto:

```

Git Bash
-----
1 | usuario@equipo MINGW64 ~/Documentos
2 | $

```

Esa línea se llama **prompt** y es la terminal diciéndote “estoy lista, escribe un comando”. Te da tres datos: quién eres (`usuario@equipo`), Capa de traducción de Unix a Windows (`MINGW64`, que es la que usa Git Bash), en qué carpeta estás (`~/Documentos`) — el símbolo `~` (Virgulilla) es una abreviatura de tu carpeta personal — y después del `$`, el espacio para escribir.

El prompt puede personalizarse para mostrar más o menos información, pero lo esencial es que siempre te indica en qué carpeta estás, lo cual es clave para no perderte ni cometer errores al ejecutar comandos.

3.2. Navegación por el sistema de archivos

En la terminal no hay iconos de carpetas: nos movemos entre directorios con comandos. La Tabla 2 reúne los esenciales; no hace falta memorizarlos ahora, vuelve a ella cuando la necesites.

Comando	¿Qué hace?	Ejemplo
<code>pwd</code>	Muestra la ruta de la carpeta actual (<i>print working directory</i>).	<code>pwd</code>
<code>ls</code>	Lista el contenido de la carpeta actual (<i>list</i>). Con <code>-la</code> muestra también los archivos ocultos y sus detalles.	<code>ls -la</code>
<code>cd</code>	Cambia de carpeta (<i>change directory</i>). <code>cd ..</code> sube a la carpeta superior.	<code>cd Documentos</code>
<code>mkdir</code>	Crea una carpeta nueva (<i>make directory</i>).	<code>mkdir proyecto</code>
<code>cp</code>	Copia archivos o carpetas (<i>copy</i>).	<code>cp datos.csv copia_datos.csv</code>
<code>mv</code>	Mueve o renombra archivos o carpetas (<i>move</i>).	<code>mv viejo.txt nuevo.txt</code>
<code>rm</code>	Elimina archivos; con <code>-r</code> elimina carpetas con todo su contenido (<i>remove</i>).	<code>rm borrador.txt</code>

Tabla 2: Comandos esenciales para navegar por el sistema de archivos.

⚠ **Atención:** en la terminal **no existe la papelera de reciclaje**. Lo que borres con `rm` desaparece de forma definitiva, así que revisa dos veces qué estás eliminando, sobre todo si usas la opción `-r`.

💡 **Consejos para escribir menos:** pulsa `Tab` para que la terminal autocomplete nombres de archivos y carpetas, y usa la flecha `↑` para recuperar los últimos comandos que escribiste.

3.3. Preparativos: instalar uv

⚠ **Atención:** Si al instalar la extensión de Python en VSCode aceptaste instalar Python automáticamente, es posible que ya tengas `uv` instalado, ya que viene incluido con las versiones recientes de Python. Para comprobarlo, abre la terminal y escribe `uv --version`. Si ves un número de versión, ¡ya lo tienes listo! En ese caso, puedes saltarte esta sección y pasar directamente a la sección 3.4.

En el ejercicio final usaremos **uv**, una herramienta moderna y muy rápida para crear entornos virtuales y gestionar bibliotecas de Python (el concepto que vimos en la sección 2.1). Para instalarla, copia el comando correspondiente a tu sistema:

💡 **Consejo:** si tu sistema es Windows, el comando de PowerShell no funcionará en la terminal de Git Bash. En ese caso, abre PowerShell (puedes buscarlo en el menú de inicio) y ejecuta el comando allí. Luego vuelve a Git Bash para continuar con el curso.

En Windows, desde PowerShell:

PowerShell

```
1 | powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

En macOS o Linux, desde la terminal:

Terminal

```
1 | curl -Lsf https://astral.sh/uv/install.sh | sh
```

⚠ **Atención:** cierra y vuelve a abrir la terminal después de instalar, para que reconozca el nuevo comando. Puedes comprobar que todo salió bien escribiendo `uv --version`: debería responder con un número de versión.

3.4. Manos a la obra: tu primer proyecto

Vamos a poner en práctica todo lo anterior creando un pequeño proyecto “Hola CV4E”: una carpeta, su entorno virtual y un programa de Python de una línea. Abre Git Bash y avancemos por fases.

Fase 1 — Orientarnos. Antes de movernos, miremos dónde estamos y qué hay alrededor:

Terminal

```
1 | pwd      # En que carpeta estoy?
2 | ls       # Que contiene esta carpeta?
```

Fase 2 — Crear la carpeta del proyecto. Nos movemos a Documentos, creamos la carpeta CV4E y entramos en ella:

Terminal

```
1 | cd Documentos
2 | mkdir CV4E
3 | cd CV4E
```

💡 **Consejo:** si tu sistema está en inglés, la carpeta se llamará `Documents`. Recuerda que con `Tab` la terminal completa el nombre por ti.

Fase 3 — Crear y activar el entorno virtual. Creamos la “placa de Petri” del proyecto con `uv` y comprobamos que existe (`ls -la` muestra los archivos ocultos, y `.venv` lo es porque empieza por punto):

Terminal

```
1 | uv venv .venv
2 | ls -la
```

Ahora lo activamos, es decir, le decimos a la terminal “a partir de ahora, trabaja dentro de este entorno”:

Terminal

```
1 | source .venv/Scripts/activate # Windows (Git Bash)
2 | source .venv/bin/activate     # macOS / Linux
```

Sabrás que funcionó porque el prompt ahora muestra `(.venv)` al principio de la línea.

Fase 4 — Crear y ejecutar el programa. Creamos un archivo de Python con una sola línea (el símbolo `>` envía el texto al archivo en lugar de mostrarlo en pantalla) y lo ejecutamos:

Terminal

```
1 | echo "print('Hola CV4E!!')" > hola.py
2 | python hola.py
```

Si todo salió bien, verás esto:

Salida esperada

```
1 | Hola CV4E!!
```

✓ ¡Felicidades! Acabas de escribir y ejecutar tu primer programa en Python, dentro de su propio entorno virtual y sin salir de la terminal.

Fase 5 — Recoger el laboratorio. Para terminar, desactivamos el entorno virtual y abrimos el proyecto en VSCode para verlo con interfaz gráfica (el punto significa “la carpeta actual”):

Terminal

```
1 | deactivate
2 | code .
```

Explora el proyecto en VSCode: en el explorador de archivos (panel 1 de la Imagen 1) verás `hola.py` y la carpeta `.venv`. Cuando termines, vuelve a la terminal y elimina el proyecto de práctica:

Terminal

```
1 | cd ..
2 | rm -r CV4E
```

⚠ **Atención:** antes de ejecutar `rm -r CV4E`, confirma con `pwd` que estás en `Documents` y no dentro de otra carpeta: recuerda que este borrado es definitivo.